

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 3, Section 3.3
- **Estimated reading time:** 15 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 1

IMPLIED VOLATILITY AND VOLATILITY SMILE

Derivative Pricing - Module 6



Outline

- ▶ Obtaining implied volatility: Newton-Raphson Method
- ▶ The volatility smile
- ▶ Volatility smile: Stylized facts
- ▶ Solutions: Advanced equity models

Introduction to Implied Volatility

We do know about the time-varying nature of volatility, but in Black-Scholes (BS), σ is constant.

Assume, for now, that BS model is valid and S_t follows a GBM. Price of a European Call is:

$$C(0, S_0) = S_0 \Phi \left(\frac{\log(S_0/K) + (r + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}} \right) - Ke^{-rT} \Phi \left(\frac{\log(S_0/K) + (r - \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}} \right) \quad (1)$$

- ▶ $C(t, S_t)$ depends on the parameters S_t , K , $T - t$, r , and σ .
- ▶ Out of these, only volatility, σ , cannot be directly observed.
- ▶ Use this equation to extract the volatility implied by the market price for exchange-traded options.
- ▶ That is the intuition behind implied volatility.

⇒ To illustrate a simple case in the next slide, focus on $t = 0$, and suppose S_0 , K , T , and r are known.

Implied Volatility: Solving for σ

We are assuming that only σ is unknown and will emphasize this by writing $C(\sigma)$.

Finding implied volatility requires solving the nonlinear problem $f(\sigma) = 0$, where:

$$f(\sigma) = S_0\Phi(d_1(\sigma)) - Ke^{-rT}\Phi(d_2(\sigma)) - C^{market}$$

for the European Call option, and:

$$f(\sigma) = Ke^{-rT}\Phi(d_2(\sigma)) - S_0\Phi(d_1(\sigma)) - P^{market}$$

for the European Put. C^{market} and P^{market} denote, respectively, the observed market values of the call and put option of interest.

► How can we solve the function above such that $f(\sigma) = 0$?

⇒ We can use recursive methods for nonlinear equations → Newton-Raphson

Implied Vol: Newton-Rapshon

First, we need the derivative of $f(\sigma)$. That is, for the Call option case:

$$f'(\sigma) = S_0 \Phi'(d_1) \frac{\partial d_1}{\partial \sigma} - Ke^{r-T} \Phi'(d_2) \frac{\partial d_2}{\partial \sigma} = S_0 \Phi'(d_1) \sqrt{T}$$

which is basically the vega of the option.

- Importantly, notice that $f'(\sigma) \geq 0$ for any σ . Thus, $f(x)$ is increasing and has at most one solution.

Next, using recursion on σ_{i+1} ,

$$\sigma_{i+1} = \sigma_i - \frac{f(\sigma_i)}{f'(\sigma_i)}, \quad \forall i \geq 0$$

Finally, we can stop the iterative process by setting a tolerance level ϵ . For instance:

$$|V^{BS} - V^{Market}| < \epsilon$$

Volatility Smile: Call Options

We could obviously repeat the process and extract implied volatility for all of the different strikes for which we have an available call option price.

- The graph that maps options' strikes with their corresponding BS implied volatility is called the **volatility smile**. The reason for the name seems obvious:

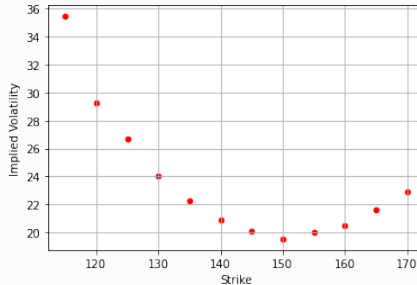


Figure: Volatility smile. IBM call options with maturity Dec, 17, 2021. Date: Oct,14,2021. IBM stock price = \$142.7

Volatility Smile: Put Options

Of course, we can follow the same exact process for put options:

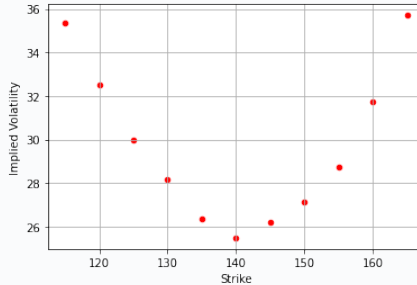


Figure: Volatility smile. IBM put options with maturity Dec, 17, 2021. Date: Oct,14,2021. IBM stock price = \$142.7

Volatility Smile: Stylized Facts

There are a few stylized facts associated with the volatility smile that are worth mentioning:

- ▶ Implied volatility of options on a single underlying varies with the strike price (and expiration time).
- ▶ We observe lower implied volatilities near the ATM strike and higher implied volatilities when we move away from the ATM strike (in both directions).
- ▶ Not all options will have an implied volatility smile:
 - Near-term equity options and currency-related options are more likely to have a volatility smile.

But remember, all of the above means a violation of the Black-Scholes model.

- ▶ In BS, volatility is constant.
- ▶ If BS perfectly described reality, we would see no smile (i.e., flat).

Volatility Smile: Are there solutions?

The Black-Scholes model fails to account for things like:

- ▶ Stock market returns have significant skewness.
- ▶ Probability of extreme events is underestimated.

All these, however, should be factored into option pricing models.

It seems we need more sophisticated models that can capture the volatility smile:

→ SDEs for price paths that recognize time-varying volatility

Which kind of models?

- ▶ Local volatility models (volatility depends on strike)
- ▶ Stochastic volatility models (e.g., Heston Model)

Summary of Lesson 1

In Lesson 1, we have looked at:

- ▶ Implied volatility
- ▶ Newton-Raphson method to obtain implied volatility
- ▶ Volatility smile and associated problems in the Black-Scholes world

⇒ **TO DO NEXT:** Now, please go to the associated Jupyter Notebook for this lesson for a more fully developed walk-through in Python of all the issues covered here.

⇒ In the next lesson, we will take a closer look at how local volatility models (e.g., Dupire) can replicate the volatility smile.

DERIVATIVE PRICING

MODULE 6 | LESSON 1

IMPLIED VOLATILITY AND VOLATILITY SMILE

Reading Time	50 minutes
Prior Knowledge	Black-Scholes, Volatility, Greeks
Keywords	Implied volatility, Volatility Smile, Newton-Raphson

In this lesson, we will guide you through the entire process of constructing the volatility smile, using data from Yahoo finance Option's chains. The option chains feature of Yahoo finance contains comprehensive data on exchange-traded options (both calls and puts) on an underlying stock. Interestingly for us at this point, it also already contains the Black-Scholes implied volatility for each of the different options and strikes.

Make sure you understand how to work with this data, as it will be an important source of information for the remainder of the course, as well as the Stochastic Modeling course. Specifically, we are going to use data on implied volatility to properly calibrate our models to market data.

So, let's get started!

1. Working with Volatility Data and the Smile

First, we are going to import the necessary libraries.

```
from datetime import date
import pandas as pd
import matplotlib.pyplot as plt
```

Next, we will work with option chain data for a particular stock—in this case, **IBM**. The spot price of IBM at the time the data was gathered was **214.23**.

There are several important considerations when working with options market data in real life applications:

- The implied volatility smile is typically more meaningful for stocks with higher volumes of exchange-traded options.
- When selecting option contracts, choose a maturity that is neither too close nor too far in the future. A good rule of thumb is to pick an expiration date roughly **three months ahead**.
- Ensure you select an exact expiration date. Since options do not expire every day, trying to use a non-existent date will result in an error.
- Implied volatility can be obtained from either call or put options. Even for the same strike, the implied volatility may differ slightly between calls and puts.

In the following example, we will use IBM's option chain data.

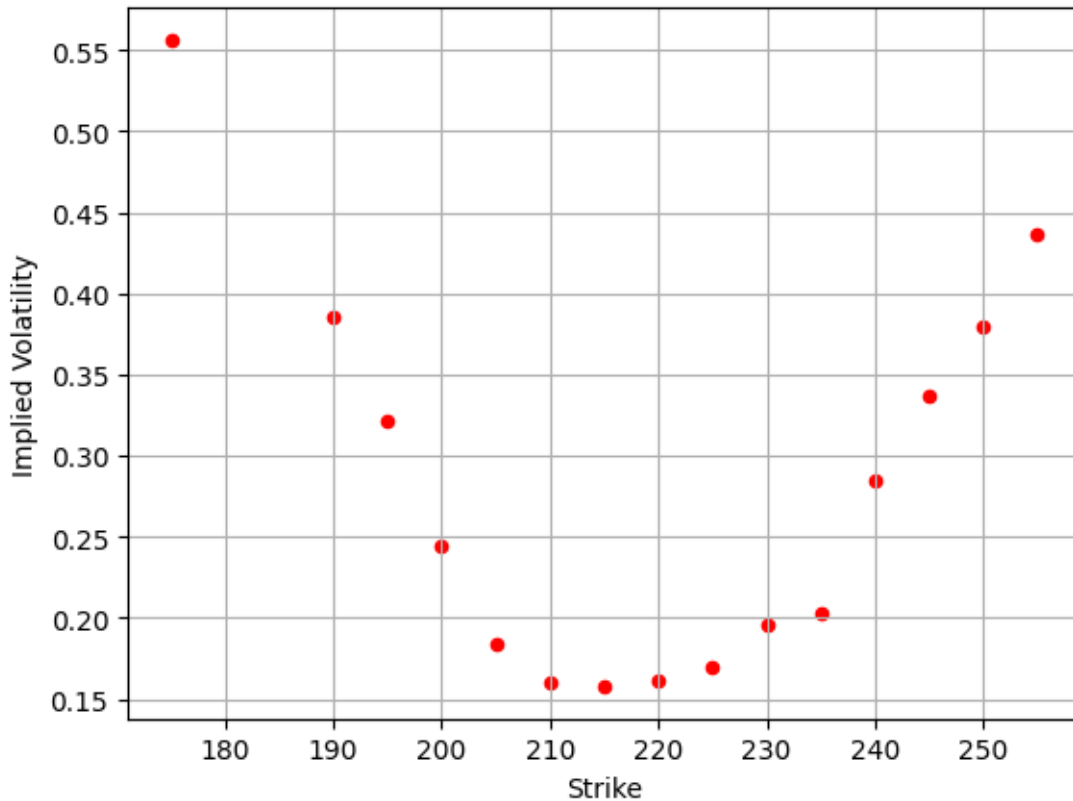
Note: The implied volatility values will typically be available only when the U.S. markets are open in many data providers.

```
chain = pd.read_excel("Options_chain.xlsx", sheet_name=['Calls', 'Puts'])
df_call=chain['Calls']
df_put=chain['Puts']
price = 214.23
```

Once we have extracted the data from the implied volatility of the call options, let's see whether there is a smile:

```
df_call = df_call[df_call["Implied Volatility"] > 0]
df_call["Strike"] = df_call["Strike"].astype(float)
df_call = df_call[
    df_call["Strike"] > price * 0.8
] # We will only consider those options within a certain range of the current price
df_call = df_call[
    df_call["Strike"] < price * 1.2
] # One reason for this is that it is precisely those options that are most often traded.
```

```
df_call.plot(kind="scatter", x="Strike", y="Implied Volatility", color="red")
plt.grid()
plt.show()
```

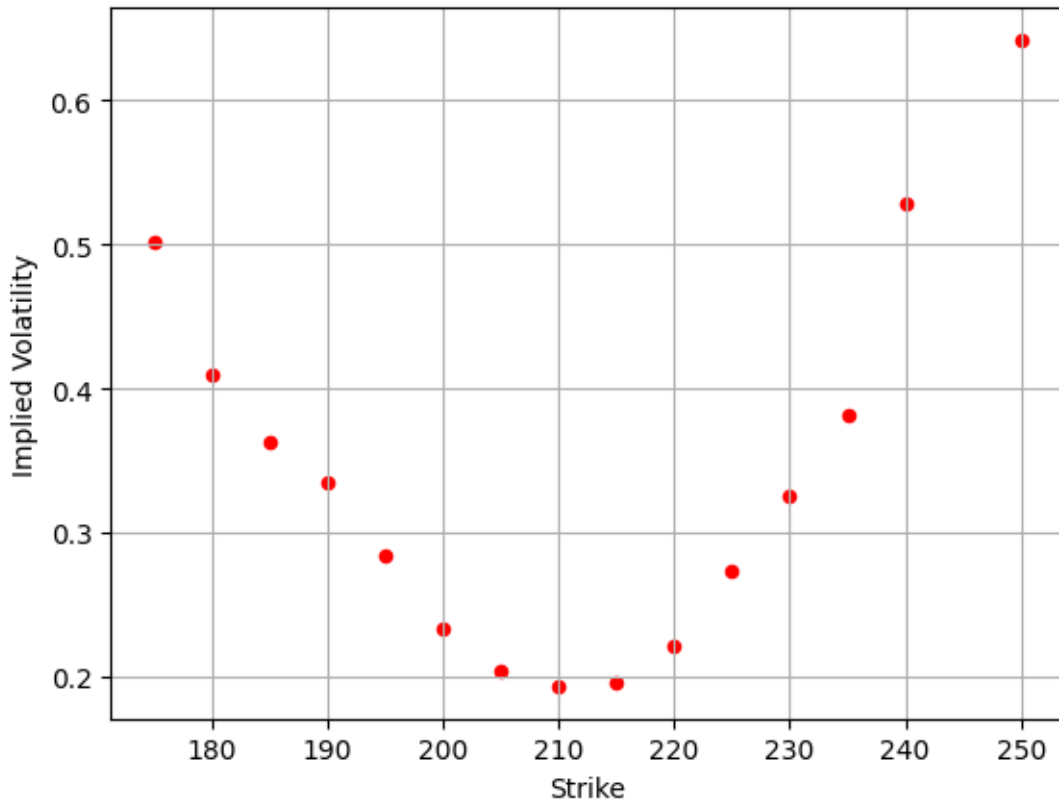


Here, you can see it is not really a complete smile. This is also called the volatility smirk (or skew). This arises when the volatility curve exhibits skewness. There are several explanations for this, mostly related to the increased demand of near ATM options from investors and put options to hedge against crashes.

And we can do the same process for put option data: All rights reserved WQU WorldQuant University QQQQ

```
df_put = df_put[df_put["Implied Volatility"] > 0]
df_put["Strike"] = df_put["Strike"].astype(float)
df_put = df_put[df_put["Strike"] > price * 0.8]
df_put = df_put[df_put["Strike"] < price * 1.2]
```

```
df_put.plot(kind="scatter", x="Strike", y="Implied Volatility", color="red")
plt.grid()
plt.show()
```



Great! You now know how to extract data regarding options' implied volatility from Yahoo finance and how to graph the volatility smile (or smirk).

2. The Newton-Raphson Method

Up to this point, we have worked with implied volatility data provided by Yahoo finance options chain data. This is essentially extracted from the BS model in the same way we have mentioned in the slides.

A convenient method to extract implied volatility data is the **Newton-Raphson recursive method**. As explained in the accompanying slides and video, this is essentially a method relying on iteration. In this last part of the notebook, we will see how to implement this method in practice.

As always, here are some necessary libraries and definitions to use later on:

```
import numpy as np
from scipy.stats import norm

N_prime = norm.pdf
N = norm.cdf
```

Then, we will need to define a few functions to simplify the task. Remember that under the Newton-Raphson method we will be iterating and updating our volatility parameter, σ , according to:

$$\sigma_{i+1} = \sigma_i - \frac{f(\sigma_i)}{f'(\sigma_i)}, \quad \forall i \geq 0$$

We'll start by defining the objective function of the optimization, $f(\sigma_i)$, the Black-Scholes pricing equation:

```
def black_scholes_call(S, K, T, r, sigma):
    """
    :param S: Asset price
    :param K: Strike price
    :param T: Time to maturity
    :param r: risk-free rate (treasury bills)
    :param sigma: volatility
    :return: call price
    """

    # Black-Scholes Formula
    d1 = (np.log(S / K) + (r + sigma**2 / 2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)

    call = S * N(d1) - N(d2) * K * np.exp(-r * T)
    return call
```

Next, we'll define $f'(\sigma_i)$, which is basically the vega of the option:

```
def vega(S, K, T, r, sigma):
    """

    :param S: Asset price
    :param K: Strike price
    :param T: Time to Maturity
    :param r: risk-free rate (treasury bills)
```

```

:param sigma: volatility
:return: partial derivative w.r.t volatility
"""

# Calculating d1 from black scholes
d1 = (np.log(S / K) + (r + sigma**2 / 2) * T) / (sigma * np.sqrt(T))
vega = S * np.sqrt(T) * N_prime(d1)
return vega

```

Finally, we build our recursive process in the `implied\_volatility\_call` function for a given level of tolerance (ϵ) and a maximum number of iterations:

```

def implied_volatility_call(C, S, K, T, r, tol=0.0001, max_iterations=100):
    """

    :param C: Observed call price
    :param S: Asset price
    :param K: Strike Price
    :param T: Time to Maturity
    :param r: risk free rate
    :param tol: error tolerance in result
    :param max_iterations: max iterations to update vol
    :return: implied volatility in percent
    """

    # assigning initial volatility estimate for input in Newton_rap procedure
    sigma = 0.3

    for i in range(max_iterations):
        # calculate difference between black-scholes price and market price with
        # iteratively updated volatility estimate
        diff = black_scholes_call(S, K, T, r, sigma) - C
        print(i)
        # break if difference is less than specified tolerance level
        if abs(diff) < tol:
            print(f"found on {i+1}th iteration")
            print(f"difference is equal to {diff}")
            break

        # use Newton-Rapshon to update the estimate
        sigma = sigma - diff / vega(S, K, T, r, sigma)

```

```
return sigma
```

So, let's check how our algorithm performs. Imagine we observe a call option with a price of 18 :
The current stock price is $S=100$; strike price $K=115$; there is one year until maturity (T); and the risk-free rate is $r=0.05$.

```
observed_price = 18
S = 100
K = 115
T = 1
r = 0.05

imp_vol = implied_volatility_call(observed_price, S, K, T, r)
print("Implied volatility using Newton Rapshon is: ", imp_vol)
```

```
0
1
2
found on 3th iteration
difference is equal to -7.274655111189077e-06
Implied volatility using Newton Rapshon is: 0.5428424065162358
```

You can check the performance of this algorithm against some of the existing (freely available) online implied volatility calculators. For example, [you can check this site](#).

Try playing around with the different parameters to see if you obtain the same numbers with your algorithm and the online calculator. Note that very small differences can arise due to tolerance levels.

3. Conclusion

Well done! Now you know a simple way to extract implied volatility from option market prices. In the next lesson, we will move on to models that allow for variability in the volatility parameters. We'll check whether we can replicate the smile with those models, or even if there is "only" a smile, that is, if the difference in implied volatility only varies with the strike term.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 3, Section 3.4
- **Estimated reading time:** 20 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 2

LOCAL VOLATILITY MODELS: DUPIRE

Derivative Pricing - Module 6



Outline

- ▶ Motivation: Local vs. Stochastic Volatility
- ▶ Local Volatility Model: Dupire
- ▶ Constructing a Local Volatility Model
- ▶ Parametric vs. Non-Parametric Models

Motivation: Local vs. Stochastic Volatility

The underlying idea behind these models is that volatility is a random variable.

- ▶ Large moves follow large moves and small moves follow small moves. $\Rightarrow$ **Volatility clustering**
- ▶ Stock returns distribution has a higher peak and fatter tails than normal distribution:
 - $\rightarrow$ These are characteristics of mixtures of distributions with different variances.

$\Rightarrow$ What is important for pricing and hedging?

- ▶ Consistency with (observed) market dynamics: More stable hedging and model parameters
- ▶ Consistency with vanilla prices: Ensure model fits risk-neutral distribution inferred from prices

Motivation: Local vs. Stochastic Volatility

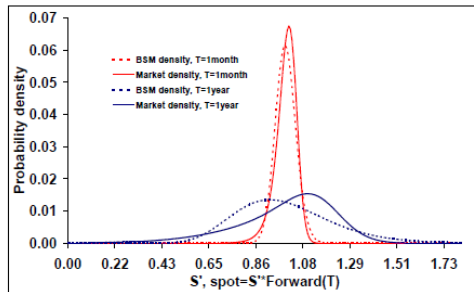
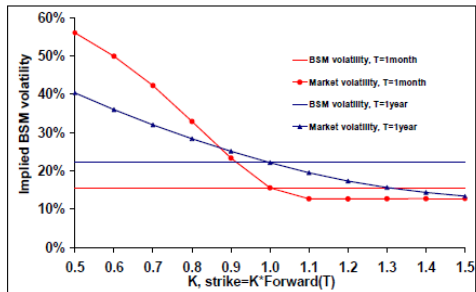


Figure: Left: Market Implied and BSM ("Black-Scholes-Merton") vol at ATM strike for $T = 1$ month and $T = 1$ year. Right: Corresponding market and BSM implied probability density.

Motivation: Stochastic Volatility

Stochastic volatility assumptions are useful because they explain in a self-consistent way why options with different strikes and maturities have different Black-Scholes implied volatilities (volatility smile).

In future lessons, we will deal with stochastic volatility models such as the following:

$$dS(t) = \mu(t)S(t)dt + \sqrt{v(t)}S(t)dW_1$$

$$dv(t) = \alpha(S, v, t)dt + \eta\beta(S, v, t)\sqrt{v(t)}dW_2$$

with

$$\langle dW_1, dW_2 \rangle = \rho dt$$

One important drawback of stochastic volatility models is their computational complexity and extreme difficulty of fitting parameters to the current vanilla prices (calibration).

Local Volatility Models

Stochastic volatility models are computationally complex and hard to calibrate in practice.

⇒ Practitioners came up with a simpler way of pricing options accounting for volatility smile: **Local Volatility**

- ▶ Dupire, Bruno. "Pricing with a Smile." *Risk*, vol. 7, no. 1, 1994, pp. 18-20.
- ▶ Derman, Emanuel, and Iraj Kani. "Riding on a Smile." *Risk*, vol. 7, no. 2, 1994, pp. 32-39.

We can distinguish between two types of local volatility models:

- ▶ **Parametric models**

Assume a parametric function for the local volatility (we will see them in the next lesson). Important for this case, parameters need to be calibrated to observed market prices.

- ▶ **Non-parametric models**

We will see an example of this next with **Dupire's model (1994)**.

Dupire's Model

Intuition: Dupire shows that there is a unique risk-neutral diffusion process (local volatility) that generates the distribution of terminal prices, S_T , conditional on S_0 .

→ Thus, we can determine the functional form of that diffusion given prices. Under the following SDE,

$$\frac{dS}{S} = \mu(t)dt + \sigma(S, t; S_0)dW$$

We can show that:

$$\sigma^2(K, T, S_0) = \frac{\frac{\partial C}{\partial T}}{\frac{1}{2}K^2 \frac{\partial^2 C}{\partial K^2}}$$

which we can compute for observable European option prices. Thus, the above equation uniquely determines local volatility as a function of option prices, strikes, and maturities.

Please refer to the additional readings/material for this lesson for a complete proof!

Constructing a Local Volatility Model

Finally, let's see how we can construct a local volatility model such as Dupire's.

Here, you have the main steps in the procedure:

1. **Assemble the necessary data:**

- Matrix of quoted option prices $C(T_i, K_{ij})$, $i = 1 \dots N$, $j = 1 \dots i$
- Yield curve data (to determine $r(t)$)
- Dividend data (to determine $q(t)$ if needed)

2. **Interpolate** and **extrapolate** these prices (i.e., corresponding BS implied volatility) to produce a smooth volatility surface.
3. Calculate volatility functional form, $\sigma^2(K, T, S_0)$ for all the observable combinations of S_0 , K , and T .
4. Finally, we can calculate the prices for other options by finite-difference methods or Monte Carlo methods.

Summary of Lesson 2

In Lesson 2, we have looked at:

- ▶ Stochastic vs. Local Volatility Models
- ▶ Local Volatility Models: Dupire
- ▶ Main Steps in Constructing Local Volatility Models

⇒ **TO-DO NEXT:** Now, please go to the associated Jupyter Notebook for this lesson in order to see how to construct a **volatility surface** in Python.

⇒ In the next lesson, we will take a closer look at parametric local volatility models and see a complete development and calibration of one type of these models (i.e., CEV model).

DERIVATIVE PRICING

MODULE 6 | LESSON 2

LOCAL VOLATILITY MODELS: DUPIRE

Reading Time	50 minutes
Prior Knowledge	Volatility Smile, Local-volatility models, Implied volatility
Keywords	Volatility surface, Dupire model

In this lesson, we will dig deeper into the nature of the volatility smile and whether the implied volatility that option market prices exhibit is determined by more variables than just strikes. All rights reserved WQU WorldQuant University QQQQ

1. Implied Volatility Surface

In the previous lesson, you learned how to construct the volatility smile extracting data from traded option prices. However, in local volatility models such as Dupire's, implied volatility from option prices is not expressed only as a function of moneyness (as in the Black-Scholes framework), but also depends on the maturity of the contract. This creates a three-dimensional dependence of implied volatility, and thus, we will move from a volatility smile to a volatility surface (3D).

The process is very similar to the previous one, with some peculiarities in terms of the amount of future maturities we consider, as well as the lessons we can learn from this exercise.

As always, let's begin by importing the necessary tools we'll need:

```

from datetime import date
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np

```

Just for comparative purposes, let's keep using the firm IBM as the focus of our investigation of volatility surface. Let's first look at the future expiration dates in the options chain data:

```

chain = pd.read_excel("Options_chain_mats.xlsx", sheet_name=['Calls', 'Puts'])
df_call=chain['Calls']
df_put=chain['Puts']
price = 214.23

```

Next, let's extract and handle the data to organize it properly:

```

today = pd.to_datetime('2024-11-11')
df_call['Maturity'] = pd.to_datetime(df_call['Maturity'], format='%Y-%m-%d')
df_put['Maturity'] = pd.to_datetime(df_put['Maturity'], format='%Y-%m-%d')

df_call['Days to maturity'] = (df_call['Maturity'] - today).dt.days
df_put['Days to maturity'] = (df_put['Maturity'] - today).dt.days

```

```
df_call
```

	Contract Name	Strike	Last Price	Bid	Ask	Change	% Change	Volume	Open Intere
0	IBM241129C00145000	145	74.40	68.30	71.40	0.00	0.0000	-	1
1	IBM241129C00150000	150	65.19	64.30	66.60	0.00	0.0000	1	1
2	IBM241129C00160000	160	46.58	53.35	55.65	0.00	0.0000	-	2
3	IBM241129C00175000	175	39.70	38.25	41.50	-17.95	-0.3114	-	7
4	IBM241129C00190000	190	17.82	24.30	25.00	0.00	0.0000	-	15
...	...	...	...	...	...	...	...	...	...
70	IBM241220C00300000	300	0.20	0.00	0.35	0.00	0.0000	3	34
71	IBM241220C00310000	310	0.31	0.00	0.38	0.00	0.0000	-	3
72	IBM241220C00315000	315	0.31	0.00	0.37	0.00	0.0000	-	1
73	IBM241220C00320000	320	0.28	0.00	0.37	0.00	0.0000	-	4
74	IBM241220C00335000	335	0.02	0.00	0.05	0.00	0.0000	10	11

Finally, after some small modifications, we can plot the data and construct our volatility surface.

Notice that in the following code snippet we are defining **moneyness** as $\text{Log}(K/S_t)$, so a moneyness of 0 refers to the ATM point.

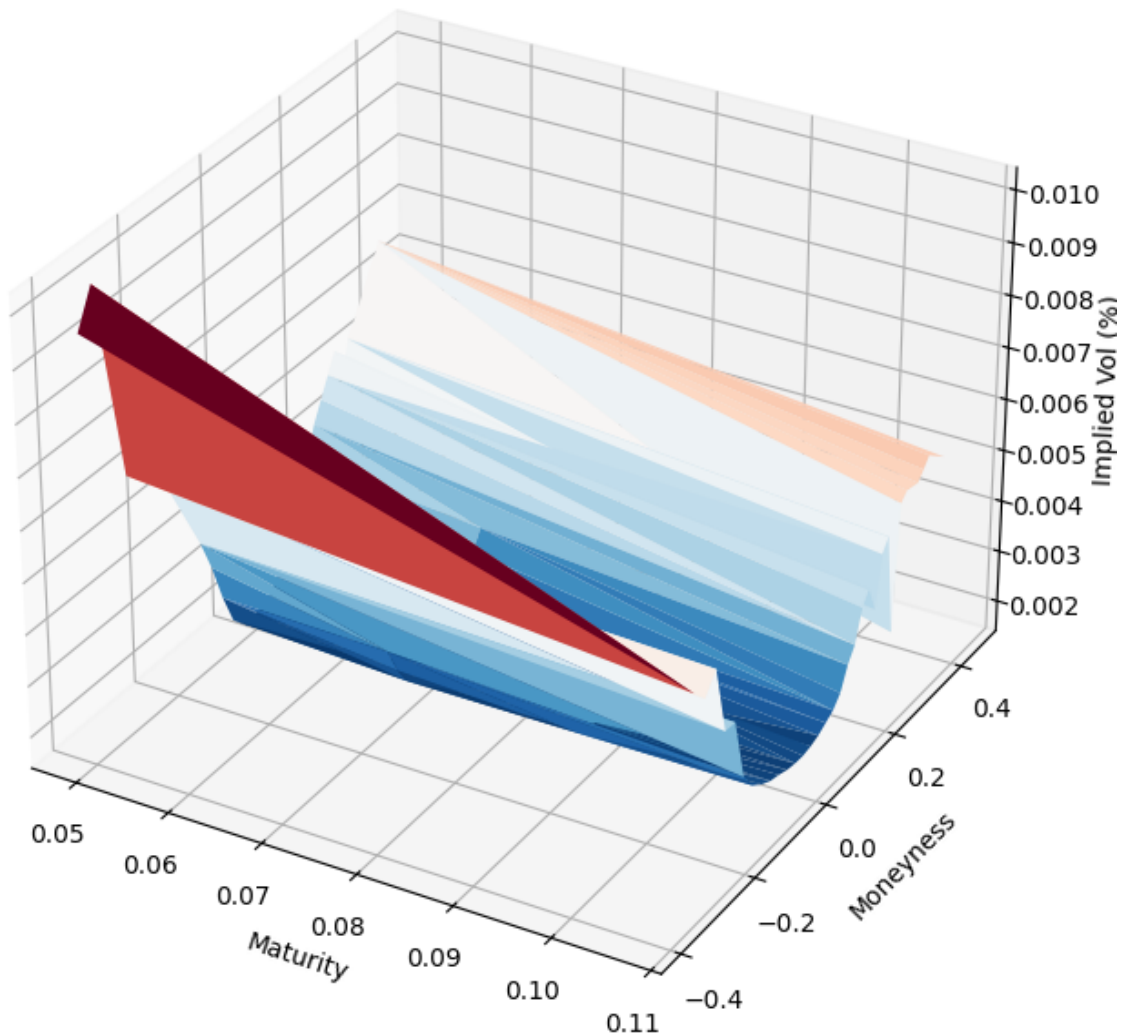
```
callData = df_call

callData = callData[callData['Implied Volatility']<90]
callData = callData[callData['Implied Volatility']>0]

fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
surf = ax.plot_trisurf(callData['Days to maturity']/365, np.log(callData['Strike']/price), c

# set axis labels
ax.set_xlabel('Maturity')
ax.set_ylabel('Moneyness')
ax.set_zlabel('Implied Vol (%)')

plt.show()
```



2. Conclusion

Now you know how to handle data to construct an implied volatility surface. If you have played around with the previous strike prices, you will have realized that the availability of options with different strikes decreases as we move away from the ATM strike. In other words, near ATM, there are options available for a wide range of strikes with little increments from one another. When we move away from this point, there are larger differences between one option's strike and the strike of the next available one. This produces certain spikes in the

volatility surface, making it less smooth than sometimes desirable (e.g., we want to price an exotic option for which there is no observable strike and, thus, implied volatility).

What can we do about this in practice?

In practice, if you want a smoother (“more continuous”) volatility surface, you will need to interpolate the volatility surface. This can become a tedious and complex process. The literature has proposed several methods to do this, although this is out of the scope of the current course. If you want to learn more about it, you can take a look at the following papers (these are **not** required readings):

- Avellaneda, Marco, et al. “Calibrating Volatility Surfaces via Relative-Entropy Minimization.” *Applied Mathematical Finance*, vol. 4, no. 1, 1997, pp. 37–64.
- Fengler, Matthias R. “Arbitrage-Free Smoothing of the Implied Volatility Surface.” *Quantitative Finance*, vol. 9, no. 4, 2009, pp. 417–428.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 3, Section 3.5
- **Estimated reading time:** 20 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 3

LOCAL VOLATILITY MODELS: CEV

Derivative Pricing - Module 6



Outline

- ▶ Local Volatility: Parametric vs. Non-Parametric Models
- ▶ Parametric Local Volatility: Constant Elasticity of Variance (CEV)

Parametric vs. Non-Parametric Local Volatility Models

In the previous lesson, we explored **Dupire's model**, an example of a non-parametric local volatility model.

There were some important benefits from the use of non-parametric models:

- ▶ We could obtain a functional form of volatility.
- ▶ This allows us to create a volatility surface.
- ▶ Volatility function uniquely determines pricing for all combos of K and T .

But also, importantly, some drawbacks:

- ▶ How to extrapolate prices to other (not traded) options: Finite differences or Monte Carlo
- ▶ What can we do to interpolate points in the volatility surface?
- ▶ Large computational costs

⇒ Parametric models are easier to implement and calibrate but require a **parametric form** for volatility.

Constant Elasticity of Variance (CEV) model

One of the classic parametric local volatility models is the constant elasticity of variance (CEV).

The CEV model assumes the following diffusion for the risk-neutral process:

$$dS = \mu(S, t)dt + \sigma(S, t)dZ$$

where,

$$\mu(S, t) = rS, \text{ and } \sigma(S, t) = \sigma S^{\beta/2}, \text{ with } 0 \leq \beta < 2$$

Given the function $\sigma(S, t)$, the **elasticity of variance** is determined by $\beta - 2$, since the return variance, $\nu(S, t)$ is,

$$\nu(S, t) = \sigma^2 S^{\beta-2} \rightarrow d\nu(S, t)/\nu(S, t) = (\beta - 2)dS/S$$

- ▶ If $\beta = 2$, elasticity is zero, and CEV model is equivalent to Black-Scholes.
- ▶ Focus on $\beta < 2 \rightarrow$ stock price & volatility relate negatively ([Campbell, 1987](#); [Glosten et al., 1993](#))
- ▶ The case of $\beta > 2$ is analyzed in [Emanuel and Macbeth \(1982\)](#).

CEV Pricing Equation

Under these dynamics, there exists a closed-form solution for the price of a call option.

- ▶ We will not go over the full derivation of the models here, just the main intuition and implementation.
- ▶ CEV models are as old as the 1980s:
 - Beckers; Shroeder; Carr and Linetsky; Gao
- ▶ Initially, CEV had large computational costs. Some papers solved that more recently:
 - Hsu et al.; Araneda and Villena
- ▶ We will follow the implementation in [Hsu et al. \(2008\)](#) Although we won't explicitly focus on the derivation of the model, you can check it in the original paper (in the lesson readings):

Hsu, Ying-Lin, T. I. Lin, and C. F. Lee. "Constant elasticity of variance (CEV) option pricing model: Integration and detailed derivation." *Mathematics and Computers in Simulation*, vol. 79, no. 1, 2008, pp. 60–71.

CEV Pricing Equation

- The call option pricing equation in Hsu et al. is, for $\beta < 2$:

$$C = S_t \left(1 - \chi^2 \left(2y; 2 + \frac{2}{2-\beta}; 2x \right) \right) - Ke^{-rt} \chi^2 \left(2x; \frac{2}{2-\beta}; 2y \right)$$

where χ^2 is the non-central Chi-square distribution function.

The other parameters are defined as (in the original paper notation):

$$y = \kappa^* K^{2-\beta}$$

$$x = \kappa^* S_t^{2-\beta} e^{r(2-\beta)t}$$

$$\kappa^* = \frac{2r}{(\sigma^2 (2-\beta) (e^{r(2-\beta)t} - 1))}$$

Summary of Lesson 3

In Lesson 3, we have looked at:

- ▶ Parametric vs. non-parametric local volatility models
- ▶ Parametric local volatility models: CEV

⇒ **TO-DO NEXT:** This lesson does not have an associated Jupyter Notebook. Instead, jump to the next lesson, where we will develop a full implementation of the CEV model, including the calibration of the model to market-implied volatilities.

⇒ In the next lesson, we will develop the CEV model in Python, as well as calibrate its parameters to observed market-implied volatilities.

LESSON 4

LOCAL VOLATILITY MODELS: CEV in Practice

Derivative Pricing - Module 6



Outline

- ▶ Implementing CEV model in Python
- ▶ Calibrate CEV model to observed option market prices

Implementing the CEV model

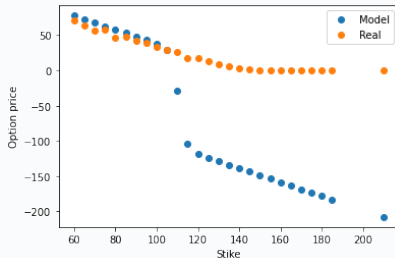
In this lesson, we are going to first learn how to implement the CEV model to price a European call option.

Using the diffusion and closed-form solution in Hsu et al., this should be a pretty easy task:

$$dS = rSdt + \sigma S^{\beta/2} dZ$$

Still, we need σ and $\beta \Rightarrow$ What can we do: Infer from the past? / Educated guess?

Picture pricing with CEV a call on IBM for $T = 3$ months, $r = 0.05$, and we assume $\sigma = 0.35$ and $\beta = 1.25$



Calibrating the CEV Model

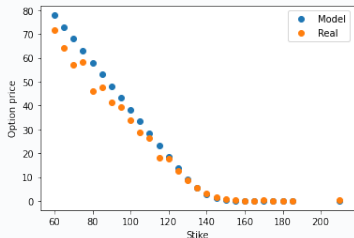
From the previous graph comparing CEV model vs. real market prices, we can extract some conclusions:

- ▶ CEV model with our choice for σ and β does a not-so-bad job for some strikes.
- ▶ It does a terrible job for other strikes.

The question at this point is can we do something about σ and β to replicate real market prices with CEV?

- ▶ Yes. We can **calibrate** the model to market prices.

⇒ In short, optimization problem to choose σ and β so that CEV output resembles market prices.



Summary of Lesson 4

In Lesson 4, we have looked at:

- ▶ Implementing CEV model
- ▶ CEV model calibration to observed option market prices

⇒ **TO-DO NEXT:** Go to the associated Jupyter Notebook for a full examination of all these issues.

⇒ In the next module, we will move on to **stochastic volatility models**, a much more realistic (but also more complex) way to model the volatility of the underlying assets.

DERIVATIVE PRICING

MODULE 6 | LESSON 4

LOCAL VOLATILITY MODELS: CEV IN PRACTICE

Reading Time	55 minutes
Prior Knowledge	Local-volatility, CEV, Calibration, Implied volatility
Keywords	CEV model, Calibration

In the previous lesson, we covered the theoretical framework for one of the most famous parametric local volatility models: the CEV (constant elasticity of variance). In this notebook, we are going to see not only how to implement the CEV model in Python but also how to properly calibrate the model to the implied volatility observed from option market prices.

As usual, let's start by importing and installing the necessary libraries.

Note: Please make sure that you gather data on options chains from Yahoo finance when the U.S. market is open. Otherwise, you may encounter errors in the code when retrieving the data.

```
import numpy as np
from scipy.stats import ncx2
from sklearn.metrics import mean_squared_error
import pandas as pd
import matplotlib.pyplot as plt
```

1. Reading Option Prices

Next, let's use the same features for extracting options chain data that we have worked on in Lessons 1 and 2 of this module. Please remember to set a maturity date for at least three months after the current date. If not, you may encounter some problems down the road related to the numerical optimization of the model/calibration (this has to do with issues related to the Jacobian).

```
chain = pd.read_excel("Options_chain_mats.xlsx", sheet_name=['Calls', 'Puts'])
df_call=chain['Calls']
df_put=chain['Puts']
price = 214.23
today = pd.to_datetime('2024-11-11')
df_call['Maturity'] = pd.to_datetime(df_call['Maturity'], format='%Y-%m-%d')
df_put['Maturity'] = pd.to_datetime(df_put['Maturity'], format='%Y-%m-%d')
df_call['Days to maturity'] = (df_call['Maturity'] - today).dt.days
df_put['Days to maturity'] = (df_put['Maturity'] - today).dt.days
df_call = df_call[df_call['Strike']>price*0.8] # We will only consider those options within
df_call = df_call[df_call['Strike']<price*1.2]

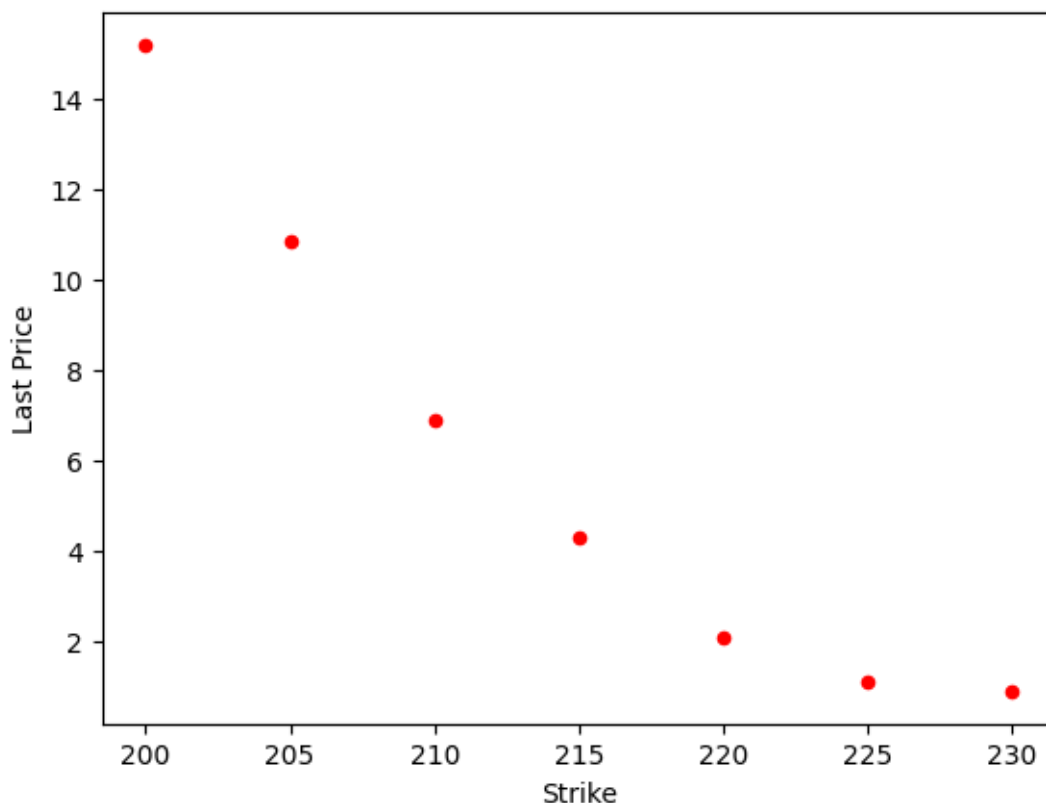
df_call = df_call[df_call['Days to maturity']==32]
```

df_call

	Contract Name	Strike	Last Price	Bid	Ask	Change	% Change	Volume	Open Interest
36	IBM241213C00200000	200	15.20	13.80	14.80	0.00	0.0000	1	3
37	IBM241213C00205000	205	10.87	10.00	10.40	0.00	0.0000	5	35
38	IBM241213C00210000	210	6.90	6.25	6.90	1.02	0.1735	3	45
39	IBM241213C00215000	215	4.30	3.80	4.35	0.40	0.1026	15	73
40	IBM241213C00220000	220	2.10	1.95	2.50	0.19	0.0995	82	148
41	IBM241213C00225000	225	1.10	0.91	2.06	0.05	0.0476	19	31
42	IBM241213C00230000	230	0.89	0.09	0.51	0.00	0.0000	4	5

With this info, let's plot the market prices for the call options as a function of the different strikes:

```
df_call.plot(kind="scatter", x="Strike", y="Last Price", color="red")
plt.show()
```



Obviously, as the option is more ITM, the premium of the call option increases. The question at this point is, can we replicate these prices with the CEV model?

2. Implementing CEV Model with Known Parameters

Next, let's code our CEV model. As we have seen in the slides from the previous lesson, there are several equivalent specifications for the CEV model. We are going to follow the one in Hsu et al., 2008. (You can check the paper in the [additional readings for the lesson](#). **Note:** This is not a required reading.)

Hsu et al.'s paper derives the following functional form for the call option price based on the following diffusion for the underlying asset:

$dS = \mu(S, t)dt + \sigma(S, t)dZ$, with:

$$\sigma(S, t) = \sigma S^{\beta/2}, \quad 0 \leq \beta < 2$$

$$\mu(S, t) = rS$$

Of course, an important assumption is going to be the choice of our parameters σ and β .

We will refine these choices later, but so far, let's just assume some given parameters. Later on, we will calibrate these parameters to market prices. For now:

- $\sigma = 0.35$
- $\beta = 1.25$

Also, let's assume a value for the risk-free rate:

- $r = 0.05$

```
import numpy as np
from scipy.stats import ncx2
from sklearn.metrics import mean_squared_error

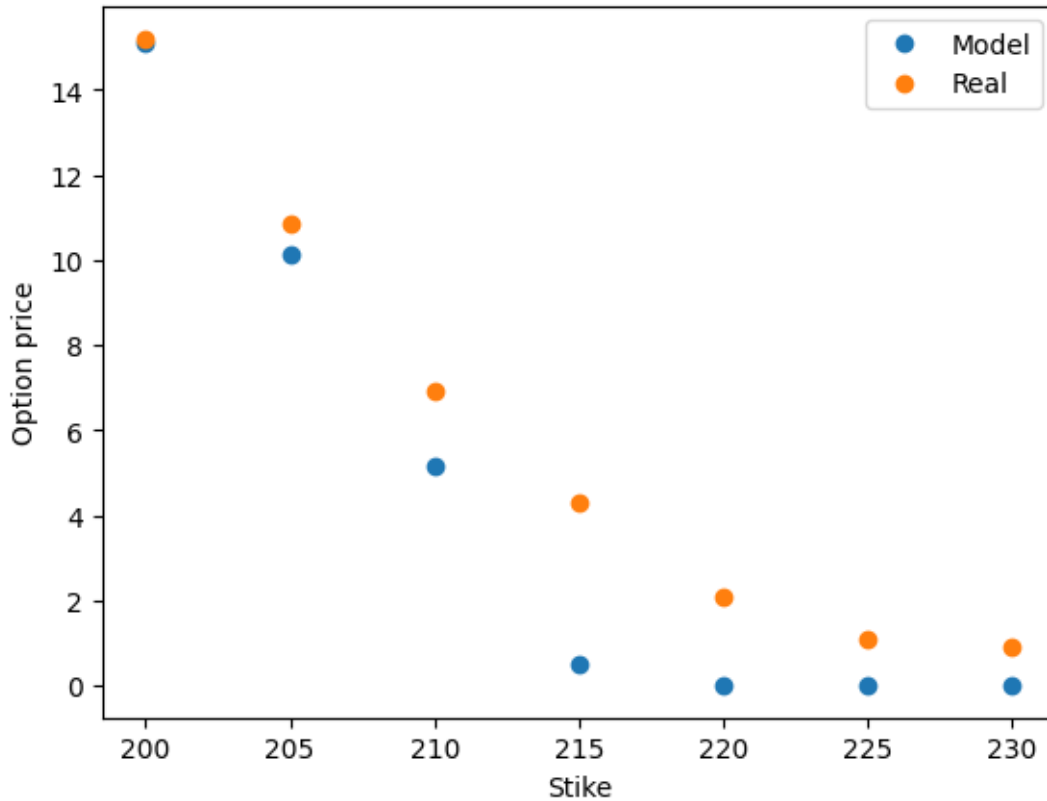
# Variables
S0 = price
r = 0.05
Td = 32/365

sigma = 0.90
beta = 0.5

def C(t,K, sigma, beta):
    zb = 2 + 2/(2-beta)
    kappa = 2*r/(sigma**2*(2-beta)*(np.exp(r*(2-beta)*t)-1))
    x = kappa*S0**(2-beta)*np.exp(r*(2-beta)*t)
    y = kappa*K**(2-beta)
    return S0*(1-ncx2.cdf(2*y,zb,2*x))-K*np.exp(-r*t)*(ncx2.cdf(2*x,zb-2,2*y))

test_strikes = df_call['Strike']
modelprices=C(Td,test_strikes, sigma, beta)
realprices = df_call['Last Price']
plt.plot(test_strikes,modelprices,'o', label='Model')
plt.plot(test_strikes,realprices,'o', label='Real')
plt.xlabel('Strike')
plt.ylabel('Option price')
plt.legend()
err = mean_squared_error(modelprices.values, realprices)
print('Mean Squared Error is ',err)
```

Mean Squared Error is 3.5007063007996493



As you can see from the previous graphs, it seems that our model is not doing a very good job in replicating the observed option prices. (You even observe some very negative option prices.) Is it because of the functional form of the model, or is it just that we did not choose our parameters wisely enough?

3. CEV Model Calibration

Finally, what we are going to do is calibrate this model to the option prices observed in the market. In other words, we are going to minimize the error between our CEV model prices and those prices observed in the market. We are going to optimize by changing only the parameters sigma (σ) and beta (β) in our CEV model. Hence, our minimization process will output the parameters sigma and beta that make the error with current market prices lower. This whole process is known as **calibration** of the model.

- Why do we only focus on these parameters? Remember, **risk-neutral valuation**.

We will import the minimize module from scipy in order to proceed with the optimization. For now, we will perform a relatively simple minimization with the default procedures in scipy. In the future, you will see that some more complex calibrations may require devoting more time

to the most suitable optimization method. All rights reserved WQU WorldQuant University
QQQQ

```
from sklearn.metrics import mean_squared_error
from scipy.optimize import minimize
```

We define our error function as the **mean squared error (MSE)** between model and market prices. This error function is what we will actually minimize.

```
def error(params):
    sigma = params[0]
    beta = params[1]

    modelprices = C(Td, test_strikes, sigma, beta) # Model price calculation using your C f
    realprices = df_call['Last Price'] # Real prices from your dataframe

    mse = mean_squared_error(modelprices, realprices)
    return mse # We want to minimize this error

bnds = ((0.2, 2), (0, 1.9)) # Example bounds, adjust as necessary
init_guess = (0.75, 0.5)

# Perform the optimization
res = minimize(error, init_guess, bounds=bnds, options={'disp': True})

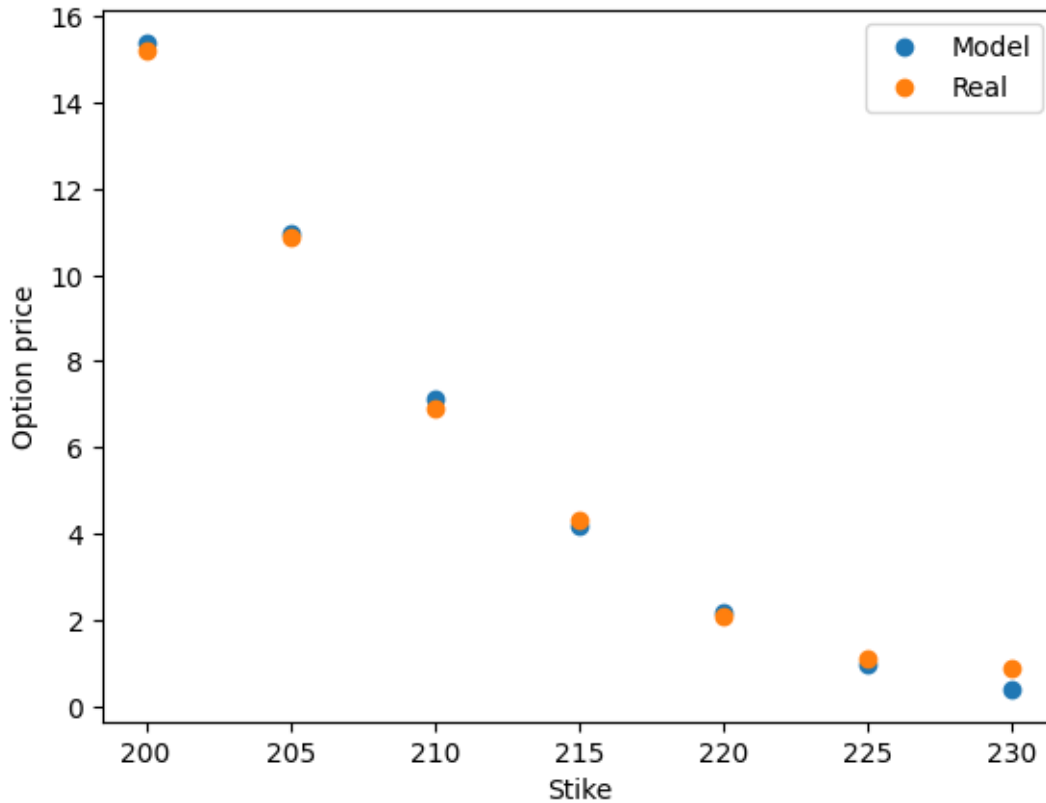
res.x[0], res.x[1]
```

```
(np.float64(0.22090021546198085), np.float64(1.8845674176933))
```

You can observe the results from the optimization process. The mean squared error (MSE) is clearly reduced from the initial scenario.

Now, let's see how the results from the model under the optimized parameters (σ and β) look in a graph versus the real market prices:

```
modelprices=C(Td,test_strikes, res.x[0], res.x[1])
realprices = df_call['Last Price']
plt.plot(test_strikes,modelprices,'o', label='Model')
plt.plot(test_strikes,realprices,'o', label='Real')
plt.xlabel('Stike')
plt.ylabel('Option price')
plt.legend()
```



4. Conclusion

Well done! Now you know how to implement a CEV model and calibrate it to option market prices. This is one of the most important jobs for a quant. In the next module, we will continue to work on these ideas, extending the framework to consider the famous stochastic volatility model of Heston.

References

- Hsu, Y. L. et al. “Constant Elasticity of Variance (CEV) Option Pricing Model: Integration and Detailed Variation.” *Mathematics and Computers in Simulation*, vol. 79, no. 1, 2008, pp. 60–71.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.